# 教案例子：亲手训练一个 2D 扩散模型

双月数据（make_moons） · 小 MLP 预测噪声 · CPU 约 20 秒出结果

配套讲义：《扩散生成模型的数学原理》讲义（diffusion_math_teaching_notes_latex.pdf）

| | |
|---|---|
| **课题** | DDPM 的前向加噪、$\epsilon$ 预测训练与逐步去噪采样（2D 可视化实验） |
| **对象** | 本科高年级 / 研究生；已学或正在学配套讲义第二、四章 |
| **先修** | Python、PyTorch 基础（张量、`nn.Module`、优化器）；高斯分布与条件概率 |
| **时长** | 90–120 分钟（概念 15 + 前向演示 20 + 训练采样 40 + 讨论 15–20，余量用于排错） |
| **硬件** | 普通笔记本 CPU 即可，无需 GPU；数据由 sklearn 内置生成，免下载 |
| **产出** | 每位学生得到：前向加噪图、loss 曲线、反向轨迹逐帧图、「数据 vs 生成」对比图 |

## 一、教学目标

完成本节课后，学生能够：

1. **前向加噪：** 用闭式 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$ 解释数据如何逐步变成近似高斯噪声，并在代码中一行实现；
2. **训练目标：** 说清网络 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ 预测的是*加进去的噪声* $\boldsymbol{\epsilon}$，而不是直接输出样本；写出 $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|^2$ 损失；
3. **反向采样：** 从 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 出发逐步去噪，生成双月分布，并读懂每一行采样代码对应的公式；
4. **实验探究：** 改变训练步数、$T$、$\beta$ 日程，观察对生成质量的影响，并与讲义 §3.2「小步长 $\Rightarrow$ 反向一步近似高斯」相联系。

**为什么这样做？ 为什么选 2D 双月而不是图像？** (1) 每个样本只是平面上一个点，整个分布一张散点图就能看全，前向「糊掉」和反向「长出来」的过程一目了然；(2) 数据由 `sklearn.datasets.make_moons` 一行生成，免下载、可复现；(3) 网络只是几层 MLP，CPU 上几十秒就能训完，课堂成功率高；(4) 双月是多峰、非高斯的分布，正好检验「从单峰高斯噪声生成多峰数据」这一核心能力。

## 二、教学准备：材料与环境

### 2.1 文件清单（随本 PDF 打包于 lab_2d_ddpm.zip）

| 文件 | 用途 |
|---|---|
| `requirements.txt` | 依赖：`torch>=2.0, numpy, matplotlib, scikit-learn` |
| `forward_demo.py` | 环节 B：前向加噪可视化（不训练，约 2 秒） |
| `classroom_ddpm.py` | 环节 C：课堂精简版（75 行，可投影 / 现场敲），见第五节 |
| `mean_collapse_demo.py` | 均值塌缩消融：MSE 回归 vs DDPM（图 6） |
| `train_2d_ddpm.py` | 完整版：命令行参数、余弦学习率、保存 ckpt 与 4 张图 |
| `runs/*/` | 本教案实测产出的图与 loss 记录（`losses.npy`） |
| `figs/` | 对比实验汇总图（训练步数 / $T$ / $\beta$） |

### 2.2 环境安装（课前完成）

CPU 版 PyTorch 的安装命令请按官方向导选择（OS / pip / CPU）：https://pytorch.org/get-started/locally/。Linux 下示例：

```
python -m venv venv && source venv/bin/activate        # Windows: venv\Scripts\activate
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install numpy matplotlib scikit-learn
python -c "import torch, sklearn; print(torch.__version__, sklearn.__version__)"
```

本教案实测环境：Linux、8 核 CPU、Python 3.13、PyTorch 2.14.1 (CPU)、scikit-learn 1.9.1。

### 2.3 运行命令

```
cd lab_2d_ddpm
python forward_demo.py                                          # 环节 B：生成 forward_demo.png
python classroom_ddpm.py                                        # 环节 C（精简版）：classroom_result.png
python train_2d_ddpm.py --T 100 --device cpu                    # 完整版，默认 400 epochs
python train_2d_ddpm.py --epochs 40 --T 100 --device cpu --out runs/moons_e40   # 对照：训练太少
```

**实测记录（本机，未经挑选，均为单次运行）。**

- `forward_demo.py`：总耗时约 2.0 s。
- `train_2d_ddpm.py --epochs 40 --T 100`：训练 2.6 s（1240 步），最终 loss（最后 50 步均值）= 0.6824；**生成结果仍是一团，看不出双月**（图 8 左）。
- `train_2d_ddpm.py --epochs 400 --T 100`：训练 22.1 s（12400 步，含采样作图共 24.7 s），最终 loss = 0.6095；生成清晰双月（图 3）。
- `classroom_ddpm.py`（400 轮，无学习率衰减）：总耗时 24.7 s，末批 loss = 0.622，生成双月（图 5）。

因此原方案中的 `--epochs 40` 只够看到 loss 下降，**不足以生成双月**；脚本默认值已改为 400（见第十节「脚本修订记录」）。

## 三、课堂流程（90–120 分钟）

### 环节 A ｜概念导入（15 分钟）

**教师活动：**在黑板画三格示意：「数据 → 噪声」（前向，固定）、「噪声 → 数据」（反向，要学）、「网络学什么」。写出第四节板书第 (1) 行，强调训练时不需要逐步加噪（闭式一步到位）。展示讲义图 0 / 图 4（总览与反向轨迹）。
**学生活动：**回答提问「如果把一张图加够多的噪声，最后还能看出原图吗?」；预测：网络的输出应该和什么同维?
**检查点：**学生能说出 $\mathbf{x}_T$ 近似 $\mathcal{N}(\mathbf{0},\mathbf{I})$，且与 $\mathbf{x}_0$ 几乎无关。

### 环节 B ｜前向演示，不训练（20 分钟）

**教师活动：**运行 `forward_demo.py`，投影图 1；指出每个子图标题里的 $\bar{\alpha}_t$。引导观察：$t=20$ 时双月仍可辨，$t=50$ 已糊成一团，$t=99$ 接近各向同性高斯。
**学生活动：**修改 `ts = [0, 20, 50, 99]` 为其他时刻；把 `beta1` 改成 `0.05` 再运行，比较 $\bar{\alpha}_{99}$。
**检查点：**学生能解释 $\sqrt{\bar{\alpha}_t}$ 是「信号剩多少」，$\sqrt{1-\bar{\alpha}_t}$ 是「噪声有多大」；并注意到 $T=100$ 时 $\bar{\alpha}_T \approx 0.36$（并非严格的纯噪声，留到讨论环节）。

### 环节 C ｜训练与采样（40 分钟）

**教师活动：**逐段讲解第五节代码（日程 → 网络 → 数据 → 训练 → 采样），对照表 1 指出每个公式落在哪一行。先让大家跑 `--epochs 40`（约 3 秒）：loss 在降，但生成是一团——引出「loss 下降 ≠ 生成好」；再跑默认 400 轮（约 25 秒）。
**学生活动：**运行 `classroom_ddpm.py` 或完整版；记录 loss；观察 `reverse_frames.png` 的逐帧演化；在代码第 64 行旁用注释写出对应公式。

**检查点：**得到双月形状的生成点云（允许毛边与少量「桥」）；能指出第 50 行就是重参数化闭式。

**环节 D ｜讨论与布置作业（15–20 分钟）**

**教师活动：**展示第七节对比实验（训练步数、$T = 20/100/200$、$\beta_{\text{end}}$），组织讨论第八节问题 1–4；布置作业。
**学生活动：**小组讨论并汇报；课后完成作业（换成 `make_circles` 或高斯混合）。
**检查点：**学生能口述「网络预测的是 $\boldsymbol{\epsilon}$」以及「为什么步子要小」。

## 四、板书三行公式

记号同讲义：$\alpha_t = 1 - \beta_t$，$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$，$\boldsymbol{\epsilon}, \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$，$t \in \{1, \dots, T\}$。

$$\text{(1) 前向（闭式）：} \quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \tag{1}$$

$$\text{(2) 训练：} \quad L(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right\|^2\right], \tag{2}$$

$$\text{(3) 反向一步：} \quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\,\mathbf{z}, \qquad \sigma_t = \sqrt{\beta_t}\ (t>1),\ \ \sigma_1 \mathbf{z} \equiv \mathbf{0}. \tag{3}$$

**为什么这样做？ 为什么只需这三行？** (1) 让训练时对任意 $t$ 一步构造 $\mathbf{x}_t$（讲义 §2.3–2.4）；(2) 是 ELBO 去权重后的简化损失（讲义式 (4.9)）；(3) 中括号部分是用网络噪声表达的反向均值 $\boldsymbol{\mu}_\theta$（讲义式 (3.43)），再加上方差 $\sigma_t^2$ 的高斯噪声——小步长下反向一步近似高斯（讲义 §3.2）。最后一步（$t = 1$）不加噪声：此时均值就是对干净数据的最终估计。

## 五、核心代码（课堂精简版，带中文注释）

下列代码即 `classroom_ddpm.py` 全文（行号与表 1 对应）。**下标约定：**代码数组从 0 开始，`betas[i]` 对应公式中的 $\beta_{i+1}$；因此代码的 `i == 0` 就是公式里的最后一步 $t = 1$。

```
# 课堂精简版 2D DDPM（双月数据，CPU 约 20 秒）——可现场敲 / 投影
import numpy as np, torch, torch.nn as nn
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from torch.utils.data import DataLoader, TensorDataset
torch.manual_seed(0)

# ---- 1. 噪声日程：beta_t、alpha_t、alpha-bar_t ----
T = 100
betas = torch.linspace(1e-4, 2e-2, T)          # beta_t，线性日程
alphas = 1 - betas                              # alpha_t = 1 - beta_t
abar = torch.cumprod(alphas, 0)                 # abar_t = prod_{s<=t} alpha_s
sa, sm = torch.sqrt(abar), torch.sqrt(1 - abar) # sqrt(abar_t), sqrt(1-abar_t)

# ---- 2. 网络 eps_theta(x_t, t)：正弦时间嵌入 + 小 MLP ----
class TimeEmb(nn.Module):
    def __init__(self, dim=32):
        super().__init__(); self.dim = dim
    def forward(self, t):                       # 整数 t -> 32 维向量
        half = self.dim // 2
        freqs = torch.exp(-np.log(10000) * torch.arange(half).float() / half)
        a = t.float().unsqueeze(1) * freqs
        return torch.cat([torch.sin(a), torch.cos(a)], -1)

class EpsNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.te = TimeEmb(32)
```

```
        self.net = nn.Sequential(
            nn.Linear(2 + 32, 128), nn.SiLU(),
            nn.Linear(128, 128), nn.SiLU(),
            nn.Linear(128, 128), nn.SiLU(),
            nn.Linear(128, 2))                  # 输出与 x 同维：预测的噪声
    def forward(self, x, t):
        return self.net(torch.cat([x, self.te(t)], -1))

# ---- 3. 数据：双月，标准化到均值 0、方差 1 ----
x, _ = make_moons(8000, noise=0.08, random_state=0)
x = torch.tensor(((x - x.mean(0)) / x.std(0)).astype("float32"))
loader = DataLoader(TensorDataset(x), batch_size=256, shuffle=True, drop_last=True)
net = EpsNet()
opt = torch.optim.AdamW(net.parameters(), lr=1e-3)

# ---- 4. 训练：随机 t、随机 eps，一步构造 x_t，回归 eps ----
EPOCHS = 400                                    # 40 轮只够 loss 下降，生成仍是一团
for epoch in range(EPOCHS):
    for (xb,) in loader:
        t = torch.randint(0, T, (xb.size(0),))  # t ~ Uniform（代码下标 0..T-1）
        eps = torch.randn_like(xb)              # eps ~ N(0, I)
        xt = sa[t].unsqueeze(1) * xb + sm[t].unsqueeze(1) * eps   # 闭式加噪（重参数化）
        loss = nn.functional.mse_loss(net(xt, t), eps)            # ||eps - eps_theta||^2
        opt.zero_grad(); loss.backward(); opt.step()
    if (epoch + 1) % 50 == 0:
        print("epoch", epoch + 1, "loss", round(float(loss), 4))

# ---- 5. 采样：从 N(0, I) 出发，逐步去噪 ----
@torch.no_grad()
def sample(n=2000):
    x = torch.randn(n, 2)                       # x_T ~ N(0, I)
    for i in reversed(range(T)):
        t = torch.full((n,), i, dtype=torch.long)
        eps = net(x, t)                         # 预测噪声
        beta, alpha, ab = betas[i], alphas[i], abar[i]
        mean = (x - beta / torch.sqrt(1 - ab) * eps) / torch.sqrt(alpha)  # mu_theta
        if i > 0:
            x = mean + torch.sqrt(beta) * torch.randn_like(x)  # + sigma_t z
        else:
            x = mean                            # 最后一步不加噪声
    return x.numpy()

ys = sample()
fig, ax = plt.subplots(1, 2, figsize=(8, 4), sharex=True, sharey=True)
ax[0].scatter(x[:2000, 0], x[:2000, 1], s=3); ax[0].set_title("data")
ax[1].scatter(ys[:, 0], ys[:, 1], s=3, c="g"); ax[1].set_title("generated")
fig.savefig("classroom_result.png", dpi=120)
```

## 5.1 公式 ↔ 代码对照表

**为什么这样做？ 为什么时间 $t$ 要做正弦嵌入再拼接？**同一个网络要处理所有噪声水平：$t$ 小时只需微调，$t$ 大时要「猜」出大部分噪声。直接输入整数 $t$ 尺度过大且难以表达；正弦 / 余弦多频率嵌入（与 Transformer 位置编码同源）给网络一组平滑、可区分的时间特征。**为什么数据要标准化？**前向过程是「方差保持」的（讲义 §2.1），只有数据本身方差约为 1，$\mathbf{x}_T$ 才接近 $\mathcal{N}(\mathbf{0},\mathbf{I})$，采样起点才与训练一致。

表 1　板书公式与 `classroom_ddpm.py` 代码行对照

| 公式 | 行号 | 代码 / 说明 |
|---|---|---|
| $\beta_t$ 线性日程，$\alpha_t = 1 - \beta_t$ | 10–11 | `betas = torch.linspace(1e-4, 2e-2, T); alphas = 1 - betas` |
| $\bar{\alpha}_t = \prod_{s \le t} \alpha_s$ | 12 | `torch.cumprod(alphas, 0)`：累乘 |
| $\sqrt{\bar{\alpha}_t}$，$\sqrt{1-\bar{\alpha}_t}$ | 13 | 预先算好，训练时按 `t` 索引 |
| $t \sim \mathrm{Uniform}\{1..T\}$ | 48 | `torch.randint(0, T, ...)`：每个样本独立抽 $t$（代码下标 0..T-1） |
| $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ | 49 | `torch.randn_like(xb)`：每次新抽，不可复用 |
| 式 (1)：$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$ | 50 | `sa[t].unsqueeze(1) * xb + sm[t].unsqueeze(1) * eps`；`unsqueeze(1)` 把 $(B,)$ 广播到 $(B, 2)$ |
| $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ | 16–35 | 正弦时间嵌入 + 4 层 MLP；输出维度 2 = 数据维度 |
| 式 (2)：$\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|^2$ | 51 | `mse_loss(net(xt, t), eps)`（对维度与批取平均，差常数倍） |
| $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ | 59 | `torch.randn(n, 2)` |
| $t = T, \ldots, 1$ | 60 | `for i in reversed(range(T))`：`i`$= t-1$ |
| 式 (3) 中的 $\boldsymbol{\mu}_\theta$ | 62–64 | `(x - beta/sqrt(1-ab)*eps) / sqrt(alpha)` |
| $+\sigma_t \mathbf{z}$，$\sigma_t = \sqrt{\beta_t}$ | 66 | `mean + torch.sqrt(beta) * torch.randn_like(x)` |
| $t = 1$ 不加噪声 | 67–68 | `i == 0` 时 `x = mean` |

## 六、预期结果（实测图）

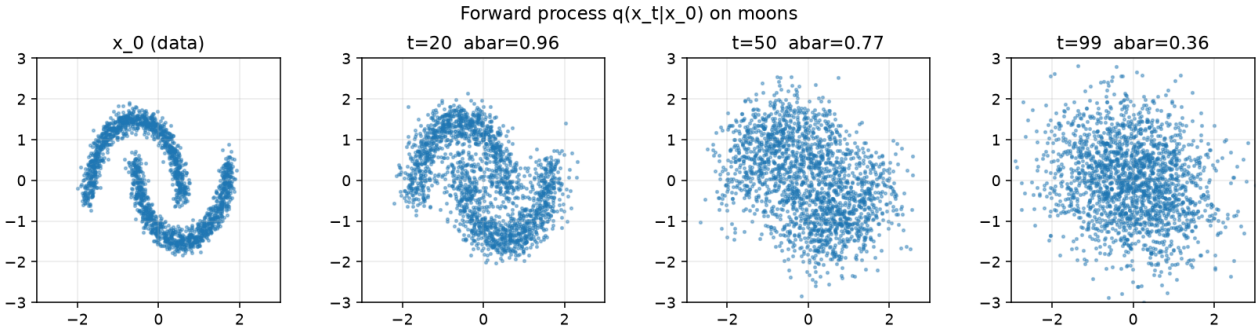



图 1　环节 B：`forward_demo.py` 实测输出。标题中的 abar 即 $\bar{\alpha}_t$（代码下标）。$t = 99$ 时 $\bar{\alpha} \approx 0.36$，点云已近似各向同性高斯。

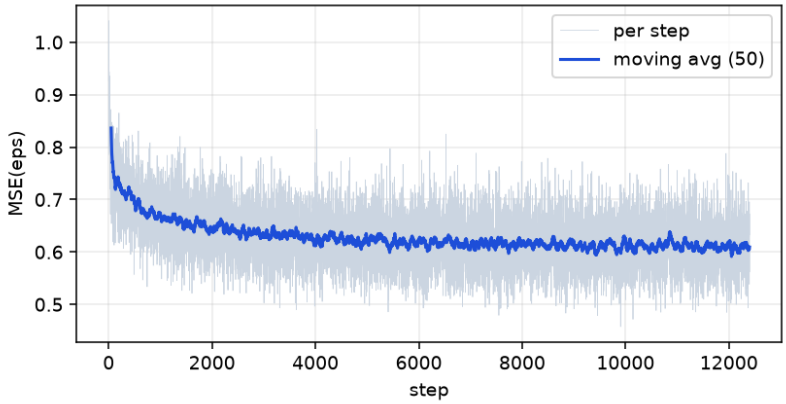



图 2　`train_2d_ddpm.py`（400 epochs，$T = 100$）的训练 loss：灰线为每步 MSE，蓝线为 50 步滑动平均。loss 很快降到约 0.6 后趋平——它不会降到 0，因为小 $t$ 时噪声几乎不可辨认（见第八节问题 2）。

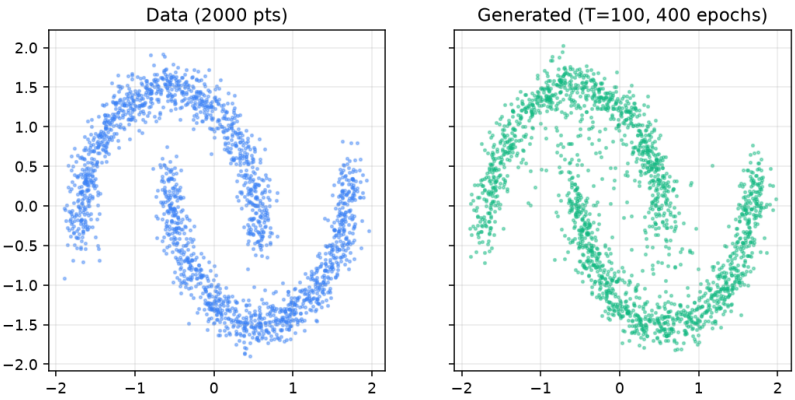



图 3　左：训练数据（2000 点）；右：模型从纯噪声生成的 2000 个点（400 epochs，$T = 100$）。

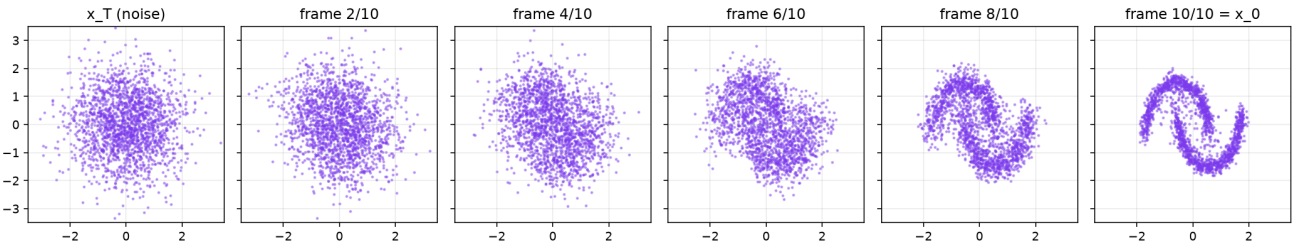



图 4　反向采样轨迹逐帧图（每 10 步记录一帧）：噪声团 → 模糊月牙 → 清晰双月。

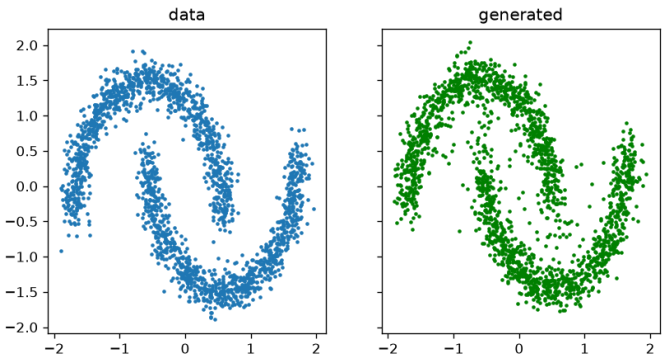



图 5　课堂精简版 `classroom_ddpm.py` 的实测输出（400 轮，约 25 s）。

## 七、为什么不能用「普通回归」生成双月？

扩散模型的训练损失也是 MSE，看起来很像「回归」。那为什么不直接训练一个网络预测 $x_0$，在 $t = T$ 时喂进噪声、一次输出 $\hat{x}_0$ 当样本？答案是：**MSE 最优解是条件均值 $E[x_0|x_t]$，多峰分布下均值落在峰与峰之间**。

**为什么这样做？** **均值塌缩（mean collapse）**。双月是双峰分布。当噪声很大（$t$ 接近 $T$）时，一个噪声点 $\mathbf{x}_t$ 几乎同样可能来自左月或右月，于是

$$E[\mathbf{x}_0 \mid \mathbf{x}_t] \;\approx\; \tfrac{1}{2}\,\mathbf{x}_{\text{左}} + \tfrac{1}{2}\,\mathbf{x}_{\text{右}}$$

落在两月之间的空隙——数据支撑集上概率几乎为 0 的地方。网络若用 MSE 回归 $\mathbf{x}_t \to \mathbf{x}_0$，学到的正是这个均值；一次去噪得到的「样本」就会堆成原点附近的一团，而不是两条月牙。
DDPM 不输出这个均值当样本，而是学习噪声 / score，再通过多步随机反向过程从 $p(\mathbf{x}_0 \mid \mathbf{x}_t)$ 里抽样——每一步只走一小步，并加上噪声，才能分别落到左月或右月。

实测消融（脚本 `mean_collapse_demo.py`；$\mathbf{x}_0$-回归器与 DDPM 使用同容量 MLP，$T = 100$）：

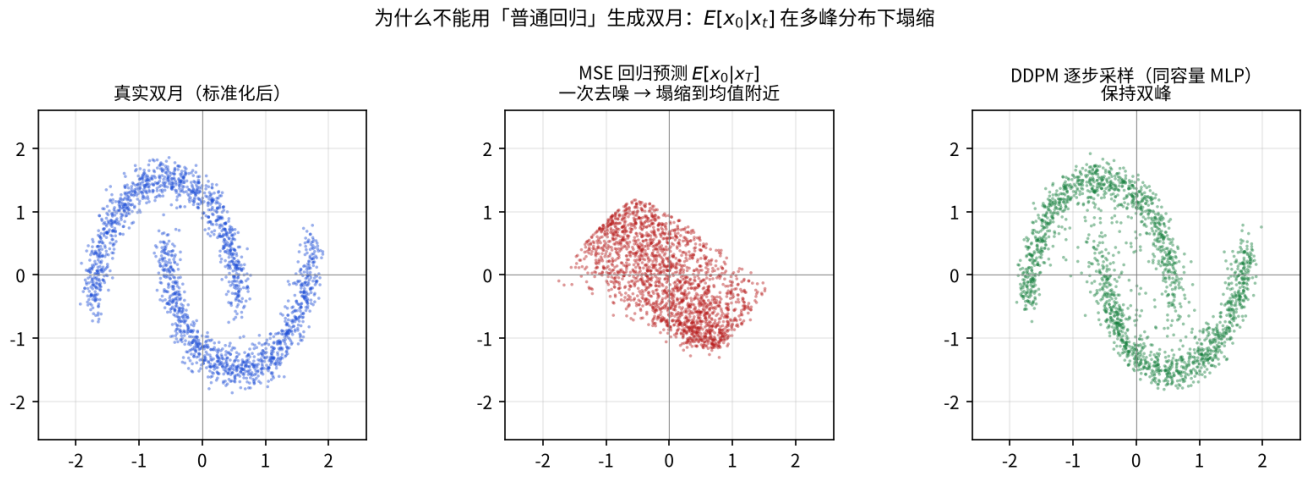



图 6　左：真实双月；中：从 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 一次回归 $\hat{\mathbf{x}}_0 = E[\mathbf{x}_0|\mathbf{x}_T]$（塌缩，均值约 $(-0.02, -0.07)$，std $\approx 0.62$）；右：同容量 DDPM 逐步采样（保持双峰，std $\approx 1.0$）。

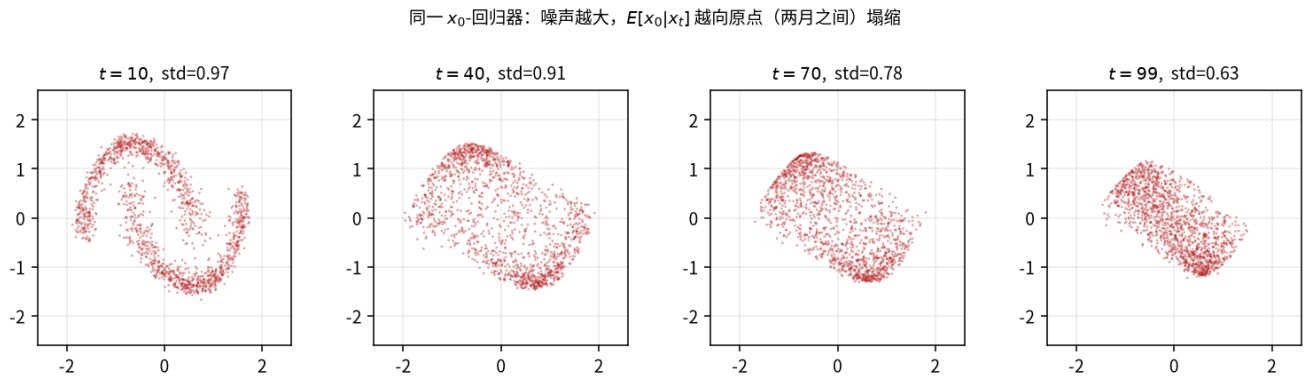



图 7　同一 $\mathbf{x}_0$-回归器在不同 $t$ 上的预测：小 $t$ 仍见双月，随噪声增大逐渐并成一团（std：0.97 → 0.91 → 0.78 → 0.63）。

**与板书公式的关系**。训练时写的是 $\mathbb{E}\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2$，其最优解等价于预测 $E[\epsilon|\mathbf{x}_t]$（或等价的 $E[\mathbf{x}_0|\mathbf{x}_t]$）。损失学到的是均值，生成靠的是采样过程——这正是「知其然又知其所以然」：MSE 负责拟合后验均值 / score，祖先采样负责从后验里抽一个样本，而不是把均值当成样本。

# 八、验收清单与对比实验

## 8.1　验收（「看到效果」）

☐ loss 曲线总体下降并趋平（图 2）；

☐ 生成点云呈双月形状（允许毛边和少量连接两月的「桥」，图 3）；

☐ 反向轨迹呈现「噪声团 → 模糊月牙 → 清晰双月」（图 4）；

☐ 能口述：网络预测的是噪声 $\epsilon$，不是直接输出样本；能指出代码中三行板书公式的位置。

## 8.2　对比实验（实测，均为单次运行）

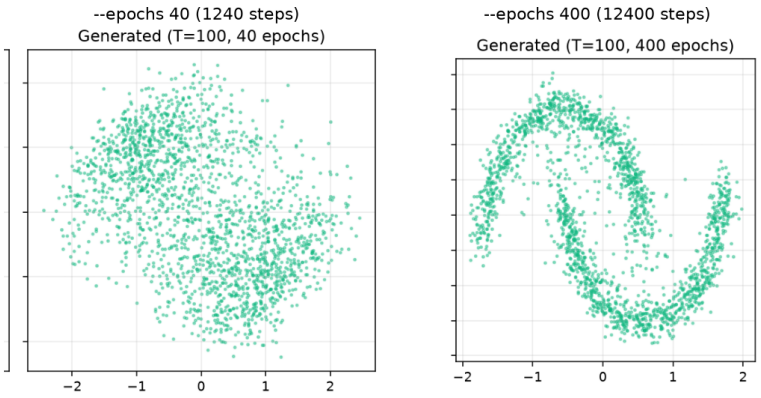



图 8　训练步数的影响（$T = 100$）：40 epochs（1240 步，loss 0.6824）只得到一团；400 epochs（12400 步，loss 0.6095）得到双月。

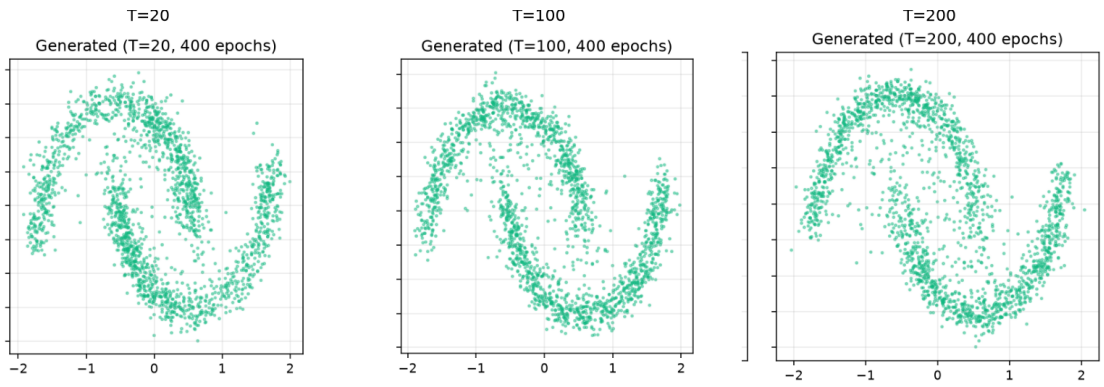



图 9　$T = 20/100/200$（$\beta$ 均为 $10^{-4} \to 0.02$，400 epochs）。实测训练时间 19.5 s / 22.1 s / 19.8 s；$\bar{\alpha}_T$ 分别为 0.82 / 0.36 / 0.13。三者都能生成双月，差别主要在毛边与「桥」的多少。

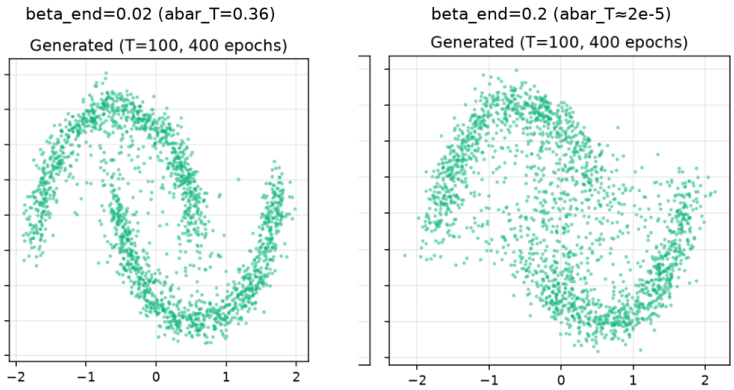



图 10　$\beta_{\text{end}} = 0.02$ 与 0.2（$T = 100$，400 epochs）。后者 $\bar{\alpha}_T \approx 2 \times 10^{-5}$（起点更像纯噪声），但每步 $\beta_t$ 大 10 倍，生成的月牙之间「桥」明显更多。

> **注意：**不同日程下 loss 数值不可直接比较（$\beta_{\text{end}} = 0.2$ 时 loss 仅 0.243，但生成更差）——因为 $t$ 的分布中「容易猜噪声」的高噪声时刻占比不同。以生成图而不是 loss 数值评价质量。

# 九、讨论问题与作业

## 9.1　课堂讨论

1. 为什么 MSE 回归 $\mathbf{x}_t \to \mathbf{x}_0$ 会得到两月之间的一团，而 DDPM 不会？（提示：最优解是 $E[\mathbf{x}_0|\mathbf{x}_t]$，生成靠采样。）
2. **loss 在降，生成却是一团（图 8）。为什么？**提示：loss 是对所有 $t$ 的平均；大 $t$ 的噪声很好猜，小 $t$ 的精细结构需要更多训练步才学得会。
3. **为什么 loss 不会降到 0？**提示：给定 $\mathbf{x}_t$，$\boldsymbol{\epsilon}$ 并不唯一确定（不同 $\mathbf{x}_0$ 都可能产生同一 $\mathbf{x}_t$）；最优网络输出的是条件期望 $\mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_t]$（讲义 §4.1 为什么框、§3.2 第 5 步）。
4. **$T = 100$ 时 $\bar{\alpha}_T \approx 0.36$，但我们从 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 出发也能生成，为什么？把 $\beta$ 放大到 0.2 让 $\bar{\alpha}_T \approx 0$ 反而更差（图 10），又为什么？**提示：数据已标准化，$\mathbf{x}_T$ 的边缘方差恰为 1、形状已接近高斯；而大 $\beta_t$ 让反向一步不再近似高斯（讲义 §3.2 第 4 步、图 5）。这是「起点误差」与「每步高斯近似误差」之间的权衡。
5. **采样时最后一步为什么不加噪声？若每步都不加噪声（$\sigma_t = 0$）会怎样？**动手试：把第 66 行的噪声项去掉，观察样本是否塌缩到月牙的「中心线」。

### 9.2 课后作业

1. $T$ **与步长：**分别用 `--T 20` 与 `--T 200` 训练（可加 `--beta-end`），把三张生成图放在一起，写 150 字分析。
2. $\beta$ **日程：**实现讲义提到的 cosine schedule（Nichol & Dhariwal 2021，arXiv:2102.09672），与线性日程对比生成质量与 $\bar{\alpha}_T$。
3. **换数据：**把 `make_moons` 换成 `make_circles`，或自己写一个 8 个高斯的混合（环形排列）；观察模型能否覆盖所有模态。
4. **采样方差：**把 $\sigma_t = \sqrt{\beta_t}$ 换成 $\sigma_t = \sqrt{\tilde{\beta}_t}$，$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$（讲义式 (3.37)），比较结果。

## 十、常见问题与排错

| 现象 | 原因与处理 |
|---|---|
| 生成是一团高斯，看不出双月 | 训练步数太少（如 `--epochs 40`，仅 1240 步）。用默认 400 轮，或减小 batch。 |
| 生成点云整体偏移 / 尺度不对 | 忘记标准化数据，或训练与采样用了不同的 `betas`。两处必须用同一套日程。 |
| 采样结果全是 NaN 或爆炸 | 公式中把 $\sqrt{1-\bar{\alpha}_t}$ 写成 $1-\bar{\alpha}_t$，或把 `sqrt(alpha)` 写成 `alpha`；检查第 64 行。 |
| `RuntimeError: shape mismatch` | `sa[t]` 形状是 $(B,)$，需 `.unsqueeze(1)` 变成 $(B,1)$ 再与 $(B,2)$ 相乘。 |
| loss 不降 | 损失目标写成了 `xb` 而不是 `eps`；或每个 batch 复用了同一个 `eps`。 |
| CPU 很慢 / 多进程卡顿 | 同时跑多个训练会争抢线程；一次只跑一个，或设置 `torch.set_num_threads(4)`。 |
| 最后一步多了一圈毛边 | $t=1$ 时（代码 `i == 0`）误加了噪声。 |

## 十一、进阶拓展

1. **MNIST + 小 U-Net：**数据见 torchvision MNIST 文档；有 GPU 更流畅，CPU 可减少通道数、少训几轮先看到模糊数字。
2. **Hugging Face Diffusers：**Diffusers 文档；官方教程 Train a diffusion model（64 分辨率蝴蝶图像，可用小 batch）。
3. **对照讲义验证「小步长」：**固定训练步数，扫描 $\beta_{\text{end}} \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$ 与 $T$，画「生成质量 vs 每步 $\beta_t$」曲线，直观验证讲义 §3.2 的结论。
4. **原论文代码：**DDPM 官方实现 hojonathanho/diffusion（TensorFlow），可对照本课代码中的日程与采样公式。

## 十二、与配套数学讲义的衔接

| 本实验环节 / 代码 | 讲义对应位置 |
|---|---|
| 环节 B、代码第 10–13、50 行 | §2.1 前向定义（式 2.1）；§2.3 闭式（式 2.4）；§2.4 重参数化；§2.5 $\beta$ 日程 |
| 代码第 51 行（损失） | §4.3 简化损失 $L_{\text{simple}}$（式 4.9）；§4.4 算法 1 |
| 代码第 62–68 行（采样） | §3.5 $\boldsymbol{\mu}_\theta$ 的 $\boldsymbol{\epsilon}$ 形式（式 3.43）；§5 算法 2 |
| 讨论问题 3（大 $\beta$ 更差） | §3.2「为什么可以把反向一步建模为高斯」第 2–4 步与图 5 |
| 讨论问题 2（loss 不为 0） | §4.1 为什么框（最优解为条件期望）；§3.2 第 5 步 |
| 作业 4（$\tilde{\beta}_t$） | §3.3 固定方差的为什么框；§3.4 后验方差（式 3.37） |

**建议顺序：**先讲讲义第二章（前向闭式）与 §4.3（$\boldsymbol{\epsilon}$-loss）→ 立即上本实验 → 回到第三章讲反向贝叶斯推导与「小步长高斯近似」，用本实验的对比图作为直观证据。

## 十三、脚本修订记录（相对原始方案）

运行原方案时发现并处理了以下问题（所有数字均来自本机实测）：

1. **训练步数不足：**原命令 `--epochs 40`（8000/256 ≈ 31 步/轮，共 1240 步）生成结果是一团。`train_2d_ddpm.py` 默认改为 400 轮（约 22 s），精简版同步改为 400 轮。
2. **缺少随机种子：**两份脚本原先结果不可复现，已加 `torch.manual_seed`。
3. **完整版新增：**余弦学习率衰减、命令行参数 `--beta-end`、训练计时与最终 loss 打印、额外输出 `data_vs_generated.png`、`reverse_frames.png`、`loss.png`。
4. **前向演示：**子图标题显示 $\bar{\alpha}_t$，让学生直接看到 $T = 100$ 时 $\bar{\alpha}_T \approx 0.36$。
5. **关于 $\bar{\alpha}_T$ 未归零：**曾尝试按 $1000/T$ 放大 $\beta$（$\bar{\alpha}_T \approx 2 \times 10^{-5}$），实测生成反而更差（图 10），故保留原线性日程，并把它作为讨论问题 3。

## 十四、参考资料与链接


[1] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. NeurIPS 2020. https://arxiv.org/abs/2006.11239

[2] DDPM 官方代码：https://github.com/hojonathanho/diffusion

[3] Y. Song et al. Score-Based Generative Modeling through Stochastic Differential Equations. ICLR 2021. https://arxiv.org/abs/2011.13456

[4] A. Nichol, P. Dhariwal. Improved Denoising Diffusion Probabilistic Models. ICML 2021. https://arxiv.org/abs/2102.09672

[5] L. Weng. What are Diffusion Models? (blog, 2021). https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

[6] PyTorch 安装向导：https://pytorch.org/get-started/locally/

[7] scikit-learn `make_moons`：https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html

[8] scikit-learn `make_circles`：https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_circles.html

[9] torchvision MNIST：https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html

[10] Hugging Face Diffusers 文档：https://huggingface.co/docs/diffusers/index

[11] Diffusers 教程 Train a diffusion model：https://huggingface.co/docs/diffusers/tutorials/basic_training


以上链接均已于 2026-10-10 逐一访问验证（HTTP 200）。